

Standards Tracking

1. **Meaningful Primitive Types:** I used int variables such as food, hydration, and protein, a double for money, and a boolean for alive.
2. **Arithmetic & Type Casting:** Arithmetic is used in walk(), while explicit type casting is used in getAverageNeeds().
3. **Custom Class Design:** My virtualAxolotl class contains private instance variables, overloaded constructors, getters, and setters.
4. **Non-Void Return Methods:** isHealthy(), getAverageNeeds(), and calculateHealthScore() return values used in conditionals.
5. **Nested or Compound Logic:** I used && and || in getConditionMessage() and checkStats().
6. **Dynamic Condition Loop:** The main simulation uses while (yourPet.getAlive()).
7. **Definite Count-Based Loop:** The play option uses a for loop for five swimming laps.
8. **Integrated Loop & Conditionals:** Conditions inside loops use break, continue, and changes to alive.

Design Trade-Offs

- I chose to use multiple methods and an option-inside-an-option system instead of putting all of the program logic inside one simple loop. For example, the user can choose the store and then select from several purchase options inside it. Although this required more methods and conditionals, it made the code more organized and gave the user more possible actions. I also added random events to make each run less predictable and more interactive.

Debugging Analysis

- The hardest bug I encountered was using == to compare Strings because I was more familiar with Python syntax. I noticed the problem when my options did not behave correctly while running the program. I fixed it by replacing == with .equals() when comparing Strings.